



PYTHON PROGRAMMING

# The Complete Masterclass Handbook

A Comprehensive Student Guide

From Zero to Advanced — 12 Chapters • Code Examples • Practice Questions



Student Edition



2026 Edition



Beginner Friendly

AUTHOR

**RahulSinghShah**

## NAVIGATE YOUR LEARNING

# Table of Contents

01

### Introduction to Python

Your first program, comments, using libraries, and how Python works

02

### Variables & Data Types

Memory model, dynamic typing, operators, input/output, and typecasting

03

### Strings

Indexing, slicing, methods, f-strings, immutability, and escape sequences

04

### Lists & Tuples

Mutable vs immutable collections, methods, slicing, and when to use each

05

### Dictionaries & Sets

Key-value pairs, hash concepts, set operations, and real-world use cases

06

### Conditional Expressions

if/elif/else, ternary operator, truthy/falsy values, and logical operators

07

### Loops

while/for loops, range(), break/continue, nested loops, and loop patterns

**08**

## **Functions & Recursions**

Defining functions, scope, \*args/\*\*kwargs, lambda, and recursive thinking

**09**

## **File I/O**

Reading/writing files, file modes, the with statement, and encoding

**10**

## **Object Oriented Programming**

Classes, objects, constructors, encapsulation, and static methods

**11**

## **Inheritance & More OOP**

Single/multiple/multilevel inheritance, super(), polymorphism, and MRO

**12**

## **Advanced Python**

Walrus operator, type hints, match-case, exceptions, comprehensions, and more

# Chapter 1: Introduction to Python

---

## What is Python?

---

- Created by **Guido van Rossum** in 1991.
- Python is an **interpreted**, high-level, general-purpose programming language.
- Its syntax is clean and close to **plain English**, making it the best first language to learn.
- Used at **Google, Netflix, Instagram, NASA, and more**.

## How Python Runs Your Code

Python is an **interpreted language**, which means your code is executed **line by line** at runtime — unlike compiled languages (C, Java) where the entire code is converted to machine code before running.

- **Compiler** → Translates the **entire program** at once into machine code, then runs it. Errors show up before execution.
- **Interpreter** → Translates and executes code **one line at a time**. If line 5 has an error, lines 1–4 still run.

This is why Python is slower than C/C++ but **much faster to develop with** — you can test ideas instantly.

## What "High-Level" Means

A high-level language **hides hardware details** from you. You don't manage memory, CPU registers, or machine instructions. Python handles all of that behind the scenes. You just write logic in human-readable form.

## The Python REPL

Python comes with an interactive shell called the **REPL** (Read-Eval-Print Loop). You can type code and see results instantly:

- **R**ead → Takes your input
- **E**val → Evaluates (runs) it
- **P**rint → Shows the result
- **L**oop → Waits for more input

Open your terminal and type `python` to enter the REPL. Great for testing small snippets quickly.

## Why Learn Python?

---

-  Easy to read and write — feels like writing instructions in English
-  Huge community and library ecosystem — over 400,000+ packages on PyPI
-  Used in Web Dev, Data Science, AI, Automation, Game Dev
-  High demand in the job market — one of the top 3 languages by every survey
-  Cross-platform — runs on Windows, macOS, Linux without changes

## 1.1 Your First Program

---

The `print()` function displays output on the screen. It is the most basic way to see what your program is doing.

```
print("Hello World")
```

You can print text, numbers, and calculations:

```
print("Welcome to Python!")  
print(42)  
print(10 + 5)      # Output: 15
```

💡 **Behavior:** `print()` adds a **newline** at the end by default. So each `print()` call outputs on a new line. You can change this with `end=""` — e.g., `print("Hello", end=" ")`.

## 1.2 Using External Libraries

---

Python has thousands of **third-party libraries** you can install via `pip` — Python's built-in package manager.

```
# Step 1: Install (run in terminal)
# pip install pyjokes

# Step 2: Use it
import pyjokes
print(pyjokes.get_joke())
```

💡 **How pip works:** When you run `pip install pyjokes`, pip downloads the package from **PyPI** (Python Package Index — [pypi.org](https://pypi.org)), installs it into your Python environment, and makes it available for `import`.

💡 **Remember:** `import` loads a library into your program. `pip install` downloads it from the internet first.

## 1.3 Comments

---

Comments are **ignored** by Python. They are notes for humans — to explain your logic, leave reminders, or temporarily disable code.

```
# This is a single-line comment

"""
This is a multi-line comment.
Use it for longer explanations.
"""
```

💡 **Best Practice:** Write comments that explain **why** you're doing something, not **what** the code does. The code itself shows what — comments should add context.

---

## Practice Questions

---

1. Write a program that prints your name, age, and city on separate lines.
  2. Print the result of `5 * 3 + 2`.
  3. What does `import` do in Python?
  4. What is the difference between an interpreter and a compiler?
  5. What does REPL stand for? Try running `2 + 2` in the Python REPL.
-



# Chapter 2: Variables & Data Types

---

## 2.1 What is a Variable?

---

A variable is a **name given to a memory location** that stores a value. Think of it as a **label on a box** — the label (variable name) points to a box (memory) that holds your data.

```
a = 30          # Variable = container to store a value
b = "harry"     # Keywords = reserved words in Python
c = 71.22       # Identifiers = class/function/variable name
```

💡 **Key Point:** Python **automatically identifies** the type of data — you don't need to declare it manually. This is called **dynamic typing**.

## How Variables Work in Memory

When you write `a = 30`, Python:

1. Creates the integer object `30` somewhere in memory
2. Creates the variable name `a` as a **reference** (pointer) to that object
3. If you later write `a = 50`, Python creates a new object `50` and points `a` to it — the old `30` is discarded

This is different from languages like C, where a variable **is** the memory box. In Python, a variable **points to** a memory box.

## Dynamic Typing vs Static Typing

- In **statically typed** languages (C, Java), you must declare the type: `int x = 5;`
- In **dynamically typed** Python, the type is determined **at runtime**: `x = 5` → Python figures out it's an `int`

This means the **same variable** can hold different types at different times:

```
x = 10          # x is int
x = "hello"     # x is now str – no error!
```

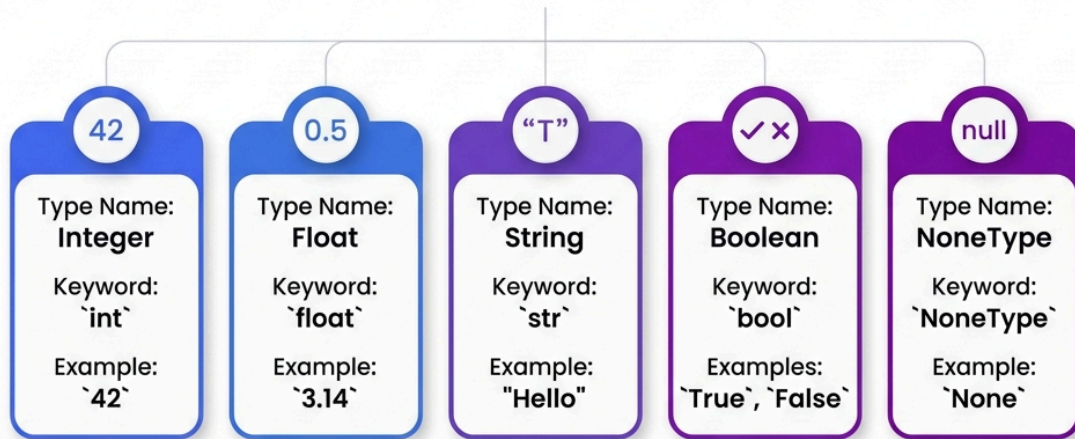
 **Be Careful:** Just because Python allows this doesn't mean you should abuse it. Changing a variable's type mid-program makes code confusing.

## 2.2 Data Types

---

Python has **5 primary data types**:

# Python Data Types



| # | Data Type | Keyword            | Example                                |
|---|-----------|--------------------|----------------------------------------|
| 1 | Integer   | <code>int</code>   | <code>71</code>                        |
| 2 | Float     | <code>float</code> | <code>88.44</code>                     |
| 3 | String    | <code>str</code>   | <code>"harry"</code>                   |
| 4 | Boolean   | <code>bool</code>  | <code>True</code> / <code>False</code> |
| 5 | None      | <code>None</code>  | <code>None</code>                      |

```
a = 71          # Python sees this as <class 'int'>
b = 88.44       # Python sees this as <class 'float'>
name = "harry"  # Python sees this as <class 'str'>
```

## Mutable vs Immutable — A Quick Intro

Every data type in Python is either **mutable** (can be changed) or **immutable** (cannot be changed after creation):

| Immutable (Cannot Change)    | Mutable (Can Change) |
|------------------------------|----------------------|
| int, float, str, bool, tuple | list, dict, set      |

When you do `a = 5` then `a = 10`, you're **not changing** the object `5` — you're creating a new object `10` and pointing `a` to it. The original `5` stays unchanged in memory.

## 2.3 Rules for Variable Names

- ✓ Can contain **alphabets**, **digits**, and **underscores**
- ✓ Can start with a **letter** or **underscore**
- ✗ **Cannot** start with a digit
- ✗ **No spaces** allowed inside a variable name
- ✗ **Cannot** use Python keywords (`if`, `for`, `while`, etc.)

**Valid Examples:** `harry`, `one8`, `_score`, `total_sum`

**Invalid Examples:**

| Variable             | Why It's Wrong                   |
|----------------------|----------------------------------|
| <code>1player</code> | ✗ Starts with a digit            |
| <code>my-name</code> | ✗ Contains a hyphen              |
| <code>@sameer</code> | ✗ Contains <code>@</code> symbol |
| <code>class</code>   | ✗ Reserved keyword               |

💡 **Naming Convention:** Use **snake\_case** for variables and functions ( `my_score` , `total_sum` ). Use **PascalCase** for class names ( `MyClass` ). This is the official Python style guide (PEP 8).

## 2.4 Operators in Python

---

### Operators in Python

#### Arithmetic Operators

`+` `-` `*` `/` `%` `**` `//`

#### Assignment Operators

`=` `+=` `-=` `*=` `/=`

#### Comparison Operators

`==` `!=` `>` `<` `>=` `<=`

#### Logical Operators

`and` `or` `not`

**1. Arithmetic Operators** — for math calculations:

```

print(10 + 3)    # Addition:      13
print(10 - 3)    # Subtraction:   7
print(10 * 3)    # Multiplication: 30
print(10 / 3)    # Division:      3.33
print(10 % 3)    # Modulus:       1 (remainder)
print(10 ** 3)   # Power:         1000
print(10 // 3)   # Floor Division: 3

```

## Operator Precedence (PEMDAS)

Python follows a **priority order** when evaluating expressions:

1. `()` — Parentheses (highest)
2. `**` — Exponentiation
3. `*`, `/`, `//`, `%` — Multiplication, Division, Floor Div, Modulus
4. `+`, `-` — Addition, Subtraction (lowest among arithmetic)

```

print(2 + 3 * 4)    # 14 (not 20! - * before +)
print((2 + 3) * 4)  # 20 (parentheses force order)

```

## 2. Assignment Operators — assign and update values:

```

x = 10    # Assign
x += 5    # Same as: x = x + 5 → 15
x -= 3    # Same as: x = x - 3 → 12
x *= 2    # Same as: x = x * 2 → 24

```

### 3. Comparison Operators — return `True` or `False` :

```
print(5 == 5)    # True    (equal to)
print(5 != 3)    # True    (not equal)
print(5 > 3)     # True    (greater than)
print(5 < 3)     # False   (less than)
```

⚠ **Common Mistake:** `=` is **assignment** ( `x = 5` stores 5 in x). `==` is **comparison** ( `x == 5` checks if x equals 5). Confusing these is one of the most common beginner bugs!

### 4. Logical Operators — combine conditions:

| Operator         | Meaning           | Example                     | Result             |
|------------------|-------------------|-----------------------------|--------------------|
| <code>and</code> | Both must be True | <code>True and False</code> | <code>False</code> |
| <code>or</code>  | At least one True | <code>True or False</code>  | <code>True</code>  |
| <code>not</code> | Reverses result   | <code>not True</code>       | <code>False</code> |

## 2.5 `type()` Function & Typecasting

`type()` tells you **what kind of data** a variable holds:



```
a = 31
type(a)      # <class 'int'>

b = "31"
type(b)      # <class 'str'>
```

**Typecasting** converts one type to another:

```
str(31)       # "31"      (int → string)
int("32")     # 32        (string → int)
float(32)     # 32.0      (int → float)
```

⚠ **Important:** `"31"` is a **string** literal. `31` is a **numeric** literal. They look similar but behave completely differently! `"31" + "9"` gives `"319"` (concatenation), while `31 + 9` gives `40` (math).

⚠ **Typecasting Error:** `int("hello")` will crash — Python can only convert strings that look like valid numbers.

## 2.6 The `input()` Function

Takes input from the keyboard. **Always returns a string** — no matter what the user types.

```
name = input("Enter your name: ")    # returns string
age = int(input("Enter age: "))       # convert to int
```

💡 **Why always string?** Because the keyboard sends characters, not numbers. When you type `25`, Python receives the characters `'2'` and `'5'` — which form the string `"25"`. You must explicitly convert it to `int` or `float` if you need a number.

---

## Practice Questions

---

1. Write a program to add two numbers entered by the user.
  2. Find the remainder when `a = 37` is divided by `b = 5`.
  3. What does `type(input("Enter: "))` return? Why?
  4. Is `a = 34` greater than `b = 80`? Write code to check.
  5. Find the average of two user-entered numbers.
  6. Calculate the square of a number entered by the user.
  7. What is the difference between `=` and `==`? Give an example.
-

# Chapter 3: Strings

---

A **string** is a sequence of characters enclosed in quotes. It is one of the most used data types in Python. Strings are **immutable** — once created, they cannot be modified in place.

## Why Strings Matter

Almost every program works with text: user names, file paths, URLs, error messages, API responses. Mastering strings is essential for real-world programming.

## 3.1 Creating Strings

---

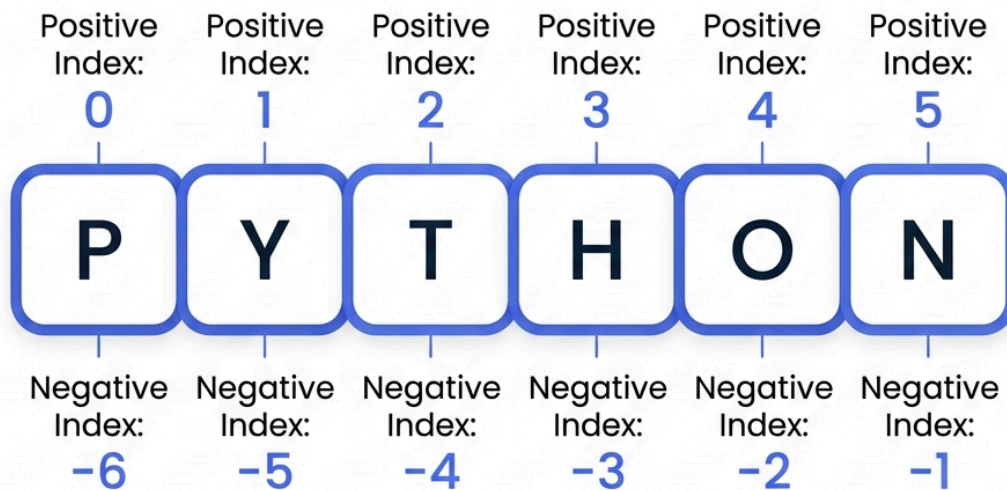
```
a = 'single quotes'
b = "double quotes"
c = '''triple quotes – for multi-line text'''
```

 **When to use which?** Single and double quotes work identically. Use one when the string contains the other: `"It's easy"` or `'He said "hello"'`. Triple quotes are for strings that span multiple lines.

## 3.2 String Indexing & Slicing

Every character has a **position (index)** starting from `0`. Negative indexing starts from the end at `-1`.

### String Indexing in Python



Name = "Python" → Length = 6

💡 **Why start at 0?** This is a convention from C. The index represents the **offset** from the start of the string. The first character is 0 positions away from the start.

**Slicing Syntax:** `string[start : stop : step]`

- `start` → index to begin (inclusive)
- `stop` → index to end (exclusive — this index is NOT included)
- `step` → how many to skip (default is 1)

```
name = "Rohan"
name[0:3]      # "Roh"  (index 0, 1, 2)
name[-4:-1]    # "oha"  (negative slicing)
name[:4]       # "Roha" (start defaults to 0)
name[1:]       # "ohan" (stop defaults to end)
name[::-1]     # "nahor" (reverse the string!)
```

 **Common Confusion:** The `stop` index is **exclusive**. `name[0:3]` gives characters at index 0, 1, 2 — NOT index 3.

## 3.3 Important String Methods

| Method                 | What It Does        | Example                                |
|------------------------|---------------------|----------------------------------------|
| <code>len(s)</code>    | Returns length      | <code>len("Python")</code> → 6         |
| <code>s.upper()</code> | All uppercase       | <code>"hello".upper()</code> → "HELLO" |
| <code>s.lower()</code> | All lowercase       | <code>"HELLO".lower()</code> → "hello" |
| <code>s.strip()</code> | Removes edge spaces | <code>" hi ".strip()</code> → "hi"     |

|                              |                       |                                                                   |
|------------------------------|-----------------------|-------------------------------------------------------------------|
| <code>s.replace(a, b)</code> | Replace a with b      | <code>"Hire".replace("H", "F")</code> →<br><code>"Fire"</code>    |
| <code>s.split(x)</code>      | Split by delimiter    | <code>"a,b,c".split(",")</code> →<br><code>['a', 'b', 'c']</code> |
| <code>s.join(list)</code>    | Join list into string | <code>"-".join(['a', 'b'])</code> → <code>"a-b"</code>            |
| <code>s.find(x)</code>       | Index of first match  | <code>"google".find("og")</code> → <code>2</code>                 |
| <code>s.count(x)</code>      | Count occurrences     | <code>"google".count("o")</code> → <code>2</code>                 |
| <code>s.startswith(x)</code> | Starts with x?        | <code>"Python".startswith("Py")</code> →<br><code>True</code>     |
| <code>s.endswith(x)</code>   | Ends with x?          | <code>"Python".endswith("on")</code> →<br><code>True</code>       |
| <code>s.capitalize()</code>  | First letter caps     | <code>"google".capitalize()</code> →<br><code>"Google"</code>     |

## 3.4 String Concatenation & F-Strings

**Concatenation** — joining strings with `+` :

```
first = "John"
last = "Doe"
full = first + " " + last    # "John Doe"
```

**F-Strings** — the modern, preferred way to embed variables (Python 3.6+):

```
name = "Alice"
age = 30
print(f"My name is {name}, I am {age} years old.")
print(f"Next year I'll be {age + 1}")    # Expressions work too
print(f"{'Hello':>10}")                  # Right-align in 10 ch
```

 **Pro Tip:** Always prefer **f-strings** over concatenation. They are faster, cleaner, and more readable. You can put **any valid expression** inside the `{}` .

**Older methods (for reference):**

```
# .format() method
print("Hello {}".format(name))
# % formatting (oldest, avoid in new code)
print("Hello %s" % name)
```

## 3.5 String Immutability

Strings **cannot be changed** after creation. Any modification creates a **new** string.

```
s = "Python"
# s[0] = 'J'           # ❌ ERROR! Strings are immutable

new_s = s.replace('P', 'J')  # ✅ Creates new string
print(new_s)                # "Jython"
print(s)                    # "Python" (original unchanged)
```

## 3.6 Escape Sequences

| Sequence        | Meaning                  |
|-----------------|--------------------------|
| <code>\n</code> | New line                 |
| <code>\t</code> | Tab space                |
| <code>\\</code> | Backslash <code>\</code> |
| <code>\'</code> | Single quote             |
| <code>\"</code> | Double quote             |



```
print("Hello\n\tWorld!")  
# Output:  
# Hello  
#      World!
```

---

## Practice Questions

---

1. Take the user's name and print `"Good Afternoon, <name>"` using an f-string.
  2. Format `"Hello John, this course is amazing. Thank you!"` using `\n` and `\t`.
  3. Check whether a sentence contains two consecutive spaces.
  4. Replace double spaces with single spaces in a sentence.
  5. Create a message template with `<name>` and `<date>` placeholders. Replace them with user input.
  6. Reverse a string entered by the user using slicing.
  7. Check if a string is a palindrome (reads the same forwards and backwards).
-

# Chapter 4: Lists & Tuples

---

## 4.1 Lists — Mutable Collections

---

A **list** is an ordered collection of items in square brackets `[]`. Lists are **mutable** — you can add, remove, and change items after creation.

### When to Use Lists

Use lists when you have a **collection of similar items** that might change: a shopping cart, student names, sensor readings, etc. Lists are the most versatile data structure in Python.

```
fruits = ["Apple", "Banana", "Cherry"]
mixed = [1, "hello", True, 3.14]      # Different types allowed
nested = [[1, 2], [3, 4]]            # Lists inside lists!
```

## Accessing & Modifying

```

fruits = ["Apple", "Banana", "Cherry"]
print(fruits[0])      # "Apple"
print(fruits[-1])     # "Cherry"

fruits[1] = "Mango"   # ✅ Lists are mutable!
print(fruits)         # ["Apple", "Mango", "Cherry"]

```

## Key List Methods

| Method                         | What It Does                                   |
|--------------------------------|------------------------------------------------|
| <code>list.append(x)</code>    | Adds <code>x</code> to the end                 |
| <code>list.insert(i, x)</code> | Inserts <code>x</code> at index <code>i</code> |
| <code>list.pop(i)</code>       | Removes & returns item at index <code>i</code> |
| <code>list.remove(x)</code>    | Removes first occurrence of <code>x</code>     |
| <code>list.sort()</code>       | Sorts in ascending order                       |
| <code>list.reverse()</code>    | Reverses the list                              |

```

nums = [3, 1, 4, 1, 5]
nums.append(9)      # [3, 1, 4, 1, 5, 9]
nums.sort()         # [1, 1, 3, 4, 5, 9]
nums.pop(0)         # Removes 1, returns it

```

## Useful Operations

```
nums = [10, 20, 30]
print(min(nums))      # 10
print(max(nums))      # 30
print(sum(nums))      # 60
print([1,2] + [3,4])  # [1, 2, 3, 4] (concatenation)
print([1,2] * 3)      # [1, 2, 1, 2, 1, 2] (repetition)
```

## 4.2 Tuples — Immutable Collections

A **tuple** is like a list but **immutable** — once created, it **cannot be changed**. Uses parentheses `()`.

### Why Use Tuples?

Tuples are **faster** than lists and signal to other programmers that this data **should not change**. Use them for fixed data like coordinates `(x, y)`, RGB colors `(255, 0, 0)`, or database records.

```
colors = ("Red", "Green", "Blue")
point = (10, 20)

# colors[0] = "Yellow"    # ❌ ERROR! Tuples are immutable
```

 **Important:** A single-element tuple needs a **comma**: `t = (5,)` not `t =`

(5) .

## Tuple Methods

Only **two** methods (because tuples can't be modified):

```
t = (1, 2, 3, 2, 2)
t.count(2)      # 3  (how many 2s)
t.index(3)      # 2  (index of first 3)
```

## 4.3 Lists vs Tuples

### List vs Tuple

**List**

[1, 2, 3]

Mutable - Can be  
changed



add, remove,  
modify

**Tuple**

(1, 2, 3)

Immutable - Cannot  
be changed



no modification

| Feature  | List <code>[ ]</code>     | Tuple <code>( )</code>  |
|----------|---------------------------|-------------------------|
| Mutable? | ✓ Yes                     | ✗ No                    |
| Speed    | Slower                    | Faster                  |
| Use Case | Shopping cart, to-do list | Coordinates, RGB colors |

Methods

Many (append, sort, etc.)

Only count, index

---



## Practice Questions

---

1. Take 4 car brand names from the user and store in a list. Display the list.
  2. Take 5 car prices, store in a list, and display in increasing order.
  3. Create a tuple of 3 car brands. Try changing one — what happens?
  4. Store prices of 4 cars in a list. Calculate total cost using `sum()`.
  5. In `cars = ("Toyota", "BMW", "Toyota", "Audi", "Toyota")`, count how many times "Toyota" appears.
-

# Chapter 5: Dictionaries & Sets

---

## 5.1 Dictionaries — Key:Value Pairs

---

A dictionary stores data as **key:value** pairs in curly braces `{}`. Keys must be **unique** and **immutable** (strings, numbers, or tuples).

### Why Dictionaries?

Dictionaries are incredibly useful when you need to **look up values by a label** rather than by position. Think of a real dictionary — you look up a word (key) to find its meaning (value). In programming, dictionaries are used for: user profiles, settings/config, API responses (JSON), counting occurrences, and caching.

```
student = {  
    "name": "Alice",  
    "age": 20,  
    "grade": "A"  
}
```

### How Dictionaries Work Internally

Dictionaries use a technique called **hashing**. When you add a key, Python calculates a **hash value** (a number) from the key, which determines where the value is stored in memory. This is why dictionary lookups are **extremely fast** —



O(1) time — regardless of how many items you have.

⚠️ **Rule:** Only **immutable** types can be dictionary keys (because the hash must not change). You can use `str`, `int`, `float`, `tuple` — but NOT `list` or `dict`.

## Accessing Values

```
print(student["name"])      # "Alice"
print(student.get("gpa"))   # None (safe — no error)
# print(student["gpa"])    # ❌ KeyError!
```

💡 **Pro Tip:** Always use `.get()` when you're unsure if a key exists. You can also provide a default: `student.get("gpa", 0)` returns `0` if key doesn't exist.

## Adding, Updating & Removing

```
student["city"] = "Mumbai"  # Add new key
student["age"] = 21         # Update existing key
del student["city"]        # Delete a key
removed = student.pop("grade") # Remove & return value
```

## Important Methods

| Method                         | Returns                |
|--------------------------------|------------------------|
| <code>d.keys()</code>          | All keys               |
| <code>d.values()</code>        | All values             |
| <code>d.items()</code>         | All (key, value) pairs |
| <code>d.update({ ... })</code> | Merge another dict     |
| <code>d.clear()</code>         | Empty the dict         |

## Dictionary Comprehension

```
squares = {x: x**2 for x in range(5)}  
# {0: 0, 1: 1, 2: 4, 3: 9, 4: 16}
```

## 5.2 Sets — Unique Elements Only

A set stores **unique**, **unordered** elements. Duplicates are automatically removed. Sets also use **hashing** internally, so membership checks ( `in` ) are very fast.

### When to Use Sets

- Removing duplicates from a list: `unique = set(my_list)`
- Checking membership quickly: `if "Alice" in my_set`

- Finding common/different items between two groups

```
my_set = {1, 2, 3, 2, 1}
print(my_set)      # {1, 2, 3} - duplicates gone!
```

⚠ **Important:** `{}` creates an **empty dictionary**, NOT a set. Use `set()` for an empty set.

## Set Operations

```
A = {1, 2, 3, 4, 5}
```

```
B = {4, 5, 6, 7, 8}
```

```
A | B      # Union:      {1,2,3,4,5,6,7,8}
```

```
A & B      # Intersection: {4, 5}
```

```
A - B      # Difference:  {1, 2, 3}
```

```
A ^ B      # Symmetric:   {1, 2, 3, 6, 7, 8}
```

# Mutable vs Immutable in Python

## Mutable (Can Change)



### List

ex. `append("5")`  
`-> append(1)`



### Dictionary

ex. `d key = 'key-value'`  
`= '2'`



### Set

ex. `set (set)`  
`-> grend(1)`

## Immutable (Cannot Change)



### String

`'str'` `-> ('str', 'st')`



### Tuple

`{1,2,3}` `-> (Sequence)`



### Integer

`10` `-> Number`



### Float

`0.06` `-> 0.00656N`

## Practice Questions

1. Create a `product_details` dictionary with `name`, `price`, and `in_stock`.  
Print the name and price.
2. Add 'History' grade to `{ 'Math': 90, 'Science': 85 }` and update 'Science' to 88.
3. Convert `['apple', 'banana', 'apple', 'grape', 'banana']` to a set to find unique fruits.

4. Find common colors between `{'red', 'blue', 'green'}` and `{'green', 'yellow', 'blue'}`.
  5. What type does `{}` create? How do you make an empty set?
  6. Can you add a list to a set? Why or why not?
-

# Chapter 6: Conditional Expressions

---

Conditionals let your program **make decisions** based on conditions. Without conditionals, your program would always do the same thing — conditionals make it **smart** and **responsive**.

## The Importance of Indentation

Python uses **indentation** (4 spaces) to define code blocks — NOT curly braces like C/Java. This is unique to Python and makes code readability a requirement, not a suggestion.

```
# Correct ✅  
if True:  
    print("Indented with 4 spaces")  
  
# Wrong ❌ – will cause IndentationError  
if True:  
print("Not indented")
```

⚠️ **Never mix tabs and spaces.** Use 4 spaces per indent level. Most editors do this automatically.

## Truthy and Falsy Values

In Python, **every value** has a truth value — not just `True` / `False` :

| Falsy (treated as <code>False</code> )                                   | Truthy (treated as <code>True</code> )                       |
|--------------------------------------------------------------------------|--------------------------------------------------------------|
| <code>False</code> , <code>0</code> , <code>0.0</code>                   | Everything else                                              |
| <code>""</code> (empty string)                                           | <code>"hello"</code> , <code>" "</code>                      |
| <code>[]</code> , <code>()</code> , <code>{}</code> , <code>set()</code> | <code>[1]</code> , <code>(1,)</code> , <code>{"a": 1}</code> |
| <code>None</code>                                                        | Any non-zero number                                          |

```

if "hello":          # Truthy – this runs
    print("Yes")

if "":               # Falsy – this is skipped
    print("No")

```

## 6.1 The `if` Statement

Executes code **only if** the condition is `True`:

```

age = 18
if age ≥ 18:
    print("You can vote!")

```

## 6.2 The `if-else` Statement

Provides an **alternative path**:

```
age = 16
if age ≥ 18:
    print("You can vote!")
else:
    print("Too young to vote.")
```

## 6.3 The `if-elif-else` Chain

---

For **multiple conditions**:

```
grade = 88

if grade ≥ 90:
    print("A")
elif grade ≥ 80:
    print("B")
elif grade ≥ 70:
    print("C")
else:
    print("F")
```

## 6.4 Nested `if`

---

An `if` **inside** another `if` :



```
num = 15
if num > 0:
    print("Positive")
    if num % 2 == 0:
        print("Even")
    else:
        print("Odd")
```

## 6.5 Ternary Operator (Shorthand if )

---

One-line conditional:

```
status = "Adult" if age ≥ 18 else "Minor"
```

## 6.6 Logical Operators in Conditions

---

```
# and — BOTH must be True
if age ≥ 18 and has_license:
    print("Can drive")

# or — at least ONE must be True
if is_weekend or has_leave:
    print("Day off!")

# not — reverses the result
if not is_hungry:
    print("Not hungry")
```

## 6.7 The `pass` Statement

---

Placeholder — does nothing but prevents syntax errors in empty blocks:

```
if condition:
    pass # TODO: implement later
```

---

## Practice Questions

---

1. Check if a number is **Even** or **Odd**.
2. Check if a user is eligible to vote (age 18+).

3. Given three sides, classify a triangle as **Equilateral**, **Isosceles**, or **Scalene**.
  4. Check if a year is a **Leap Year**.
  5. Simulate a traffic light — input a color and print the action.
  6. Build a simple login system with `username` and `password` checks.
  7. What does `if []:` evaluate to? What about `if [0]:`? Why?
  8. Rewrite this using a ternary operator: 

```
if x > 0: result = "positive"
else: result = "negative"
```
-

# Chapter 7: Loops

---

Loops **repeat** a block of code multiple times — essential for processing data and automating tasks.

## Why Loops Matter

Without loops, if you wanted to print numbers 1 to 1000, you'd need 1000 `print()` statements. Loops let you do it in 3 lines. They are the backbone of:

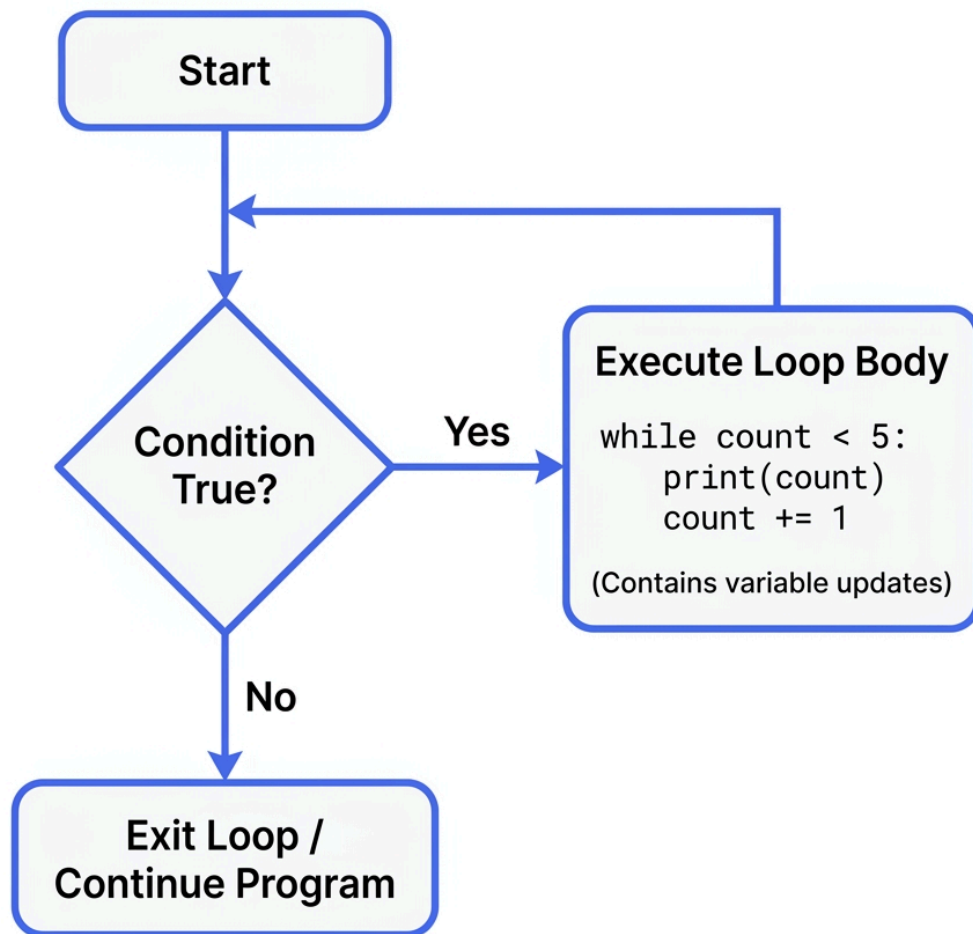
- Data processing (reading thousands of records)
- Automation (repeating tasks)
- Game loops (continuous game state updates)
- Searching and sorting algorithms

## 7.1 The `while` Loop

---

Repeats as long as the condition is `True`. Use `while` when you **don't know** how many times you need to loop.

## How a While Loop Works



```
i = 1
while i ≤ 5:
    print(i)
    i += 1      # Without this → infinite loop!
```

**⚠ Infinite Loops:** If the condition **never becomes False**, the loop runs forever and your program freezes. Always ensure there's a way out!

## 7.2 The `for` Loop

Iterates over a sequence (list, string, range, etc.). Use `for` when you **know** the collection to iterate over.

```
fruits = ["Apple", "Banana", "Cherry"]
for fruit in fruits:
    print(fruit)
```

💡 **for vs while:** Use `for` when iterating over a known collection or range. Use `while` when looping until a condition changes (e.g., user input, game state).

## 7.3 The `range()` Function

Generates a sequence of numbers:

| Syntax                       | Output        |
|------------------------------|---------------|
| <code>range(5)</code>        | 0, 1, 2, 3, 4 |
| <code>range(2, 6)</code>     | 2, 3, 4, 5    |
| <code>range(1, 10, 2)</code> | 1, 3, 5, 7, 9 |
| <code>range(5, 0, -1)</code> | 5, 4, 3, 2, 1 |

```
for i in range(1, 6):  
    print(i)      # Prints 1, 2, 3, 4, 5
```

## 7.4 Loop Control Statements

| Statement             | Effect                            |
|-----------------------|-----------------------------------|
| <code>break</code>    | <b>Exits</b> the entire loop      |
| <code>continue</code> | <b>Skips</b> current iteration    |
| <code>pass</code>     | Does <b>nothing</b> (placeholder) |

```
# break – exit when i is 5  
for i in range(1, 10):  
    if i == 5:  
        break  
    print(i)      # 1, 2, 3, 4  
  
# continue – skip even numbers  
for i in range(1, 6):  
    if i % 2 == 0:  
        continue  
    print(i)      # 1, 3, 5
```

## 7.5 For-Else Loop

The `else` block runs **only if no** `break` occurred. This is an unusual Python feature — very useful for search algorithms:

```
# Search for a number
for n in [1, 3, 5, 7]:
    if n == 4:
        print("Found!")
        break
else:
    print("Not found in the list")    # This runs!
```

## Nested Loops

A loop inside another loop. The inner loop runs **completely** for each iteration of the outer loop:

```
for i in range(1, 4):          # Outer: runs 3 times
    for j in range(1, 4):      # Inner: runs 3 times per outer
        print(f"{i}×{j}={i*j}", end="\t")
    print()                    # New line after each row
```

 **Performance Note:** Nested loops multiply iterations. 2 loops of 1000 = 1,000,000 operations. Be careful with large data!



## Practice Questions

---

1. Print the **multiplication table** of a user-entered number.
  2. Find the **sum of first N natural numbers** using a `while` loop.
  3. From `["Sahil", "Rohan", "Sameer", "Rahul"]`, greet only names starting with `'S'`.
  4. Calculate the **factorial** of a number using a `for` loop.
  5. Check if a number is **prime** or not.
  6. Print the star pattern: `*`, `**`, `***` for N rows.
-

# Chapter 8: Functions & Recursions

---

## 8.1 What is a Function?

---

A function is a **reusable block of code** defined with the `def` keyword. Functions are the building blocks of any well-organized program.

### Why Functions?

- **DRY Principle** — Don't Repeat Yourself. Write once, use many times.
- **Modularity** — Break complex problems into small, manageable pieces.
- **Readability** — `calculate_tax(income)` is clearer than 15 lines of math.
- **Testability** — Small functions are easy to test and debug.

```
def greet(name):  
    print(f"Hello, {name}!")  
  
greet("Rahul")      # Hello, Rahul!  
greet("Sameer")     # Hello, Sameer!
```

## Function Anatomy

```
def function_name(parameters):    # Header
    """Docstring – describes what it does"""    # Optional but
    # Body – the logic
    return result                # Optional – sends value ba
```

## 8.2 Arguments & Return Values

---

```
def add(a, b):
    return a + b

result = add(10, 20)
print(result)          # 30
```

## 8.3 Default Parameters

---

If no argument is passed, the default value is used:

```
def greet(name="Student"):
    print(f"Hello, {name}")

greet()          # Hello, Student (default)
greet("Tom")     # Hello, Tom (provided)
```

## 8.4 \*args and \*\*kwargs

---

Accept a **variable number** of arguments:

```
# *args – collects extra positional args as a tuple
def add_all(*args):
    return sum(args)

print(add_all(1, 2, 3, 4))    # 10

# **kwargs – collects extra keyword args as a dictionary
def show_info(**kwargs):
    for key, value in kwargs.items():
        print(f"{key}: {value}")

show_info(name="Alice", age=25)    # name: Alice, age: 25
```

💡 **Naming:** `args` and `kwargs` are conventions — you could name them anything. The `*` and `**` are what matter.

## 8.5 Lambda Functions

---

Anonymous, one-line functions:

```

square = lambda x: x ** 2
print(square(5))      # 25

# Useful with sort, map, filter:
nums = [3, 1, 4, 1, 5]
nums.sort(key=lambda x: -x)    # Sort descending

```

## 8.6 Scope — Local vs Global

Variables defined **inside** a function are **local** — they don't exist outside:

```

def my_func():
    x = 10      # Local — only exists inside my_func
    print(x)

my_func()
# print(x)    # ❌ NameError! x doesn't exist here

```

💡 **LEGB Rule:** Python searches for variables in this order: **Local** → **Enclosing** → **Global** → **Built-in**.

## 8.7 Recursion

A function that **calls itself**. Every recursive function must have:

1. **Base case** — condition to stop (prevents infinite recursion)

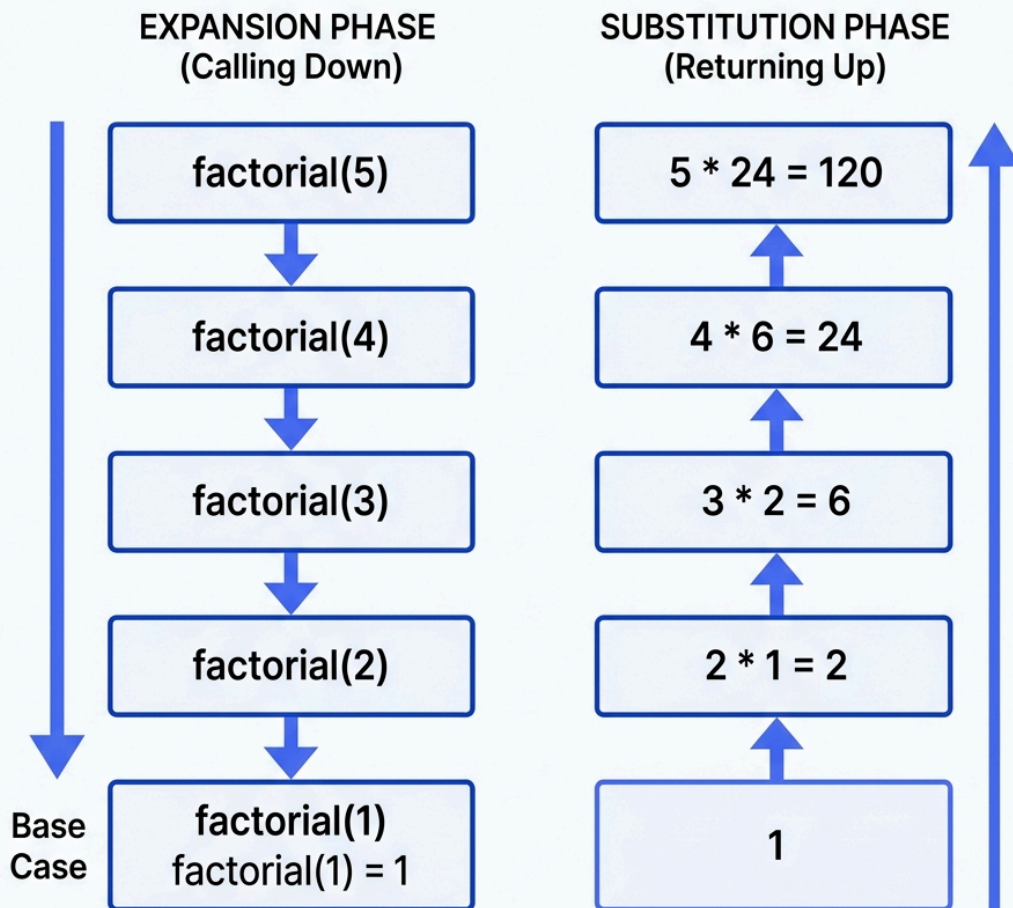
2. **Recursive case** — the function calls itself with a simpler input

```
def factorial(n):  
    if n == 0 or n == 1:    # Base case  
        return 1  
    return n * factorial(n - 1)    # Recursive call  
  
print(factorial(5))    # 120
```

 **Recursion Limit:** Python has a default recursion limit of ~1000. For very deep recursion, use loops instead or increase the limit with `sys.setrecursionlimit()`.

## How It Works — Step by Step

### How Recursion Works - `factorial(5)`



#### Going Down (Expansion):

- `factorial(5)` → `5 × factorial(4)`
- `factorial(4)` → `4 × factorial(3)`
- `factorial(3)` → `3 × factorial(2)`
- `factorial(2)` → `2 × factorial(1)`
- `factorial(1)` → `1` ← **Base Case!**

## Coming Back Up (Substitution):

- $2 \times 1 = 2 \rightarrow 3 \times 2 = 6 \rightarrow 4 \times 6 = 24 \rightarrow 5 \times 24 = 120$
- 

## Practice Questions

---

1. Write `find_max(a, b, c)` — returns the largest of three numbers.
  2. Write a function to convert Celsius to Fahrenheit:  $F = (C \times 9/5) + 32$  .
  3. Write a **recursive** function for the sum of first N natural numbers.
  4. Write `print_pattern(n)` that prints an inverted star triangle.
  5. Write a function to convert inches to centimeters (1 inch = 2.54 cm).
  6. Write a function to remove a specific word from a string and strip extra spaces.
-



# Chapter 9: File I/O

---

Files let you **store data permanently** on disk — beyond the life of your program. Without file I/O, all your data disappears when the program ends.

## Real-World Uses

- Saving user settings and preferences
- Reading CSV data for analysis
- Writing log files for debugging
- Creating reports and exports

## 9.1 File Modes

### File Open Modes in Python



| Mode | Action                               | If File Missing |
|------|--------------------------------------|-----------------|
| 'r'  | Read only                            | ✗ Error         |
| 'w'  | Write ( ⚠ <b>erases</b> everything!) | Creates new     |
| 'a'  | Append (safe — adds to end)          | Creates new     |

`'r+'`

Read and write

 Error`'rb'`

Read binary (images, PDFs)

 Error

**CRITICAL:** `'w'` mode **immediately wipes** all existing content the moment the file is opened. Use `'a'` if you want to keep old data.

## 9.2 Writing & Reading

```
# Writing
f = open("notes.txt", "w")
f.write("Hello World\n")
f.close()

# Reading
f = open("notes.txt", "r")
print(f.read())
f.close()
```

## 9.3 The `with` Statement (Best Practice)

Automatically closes the file — even if an error occurs. This is called a **context manager**:

```
with open("notes.txt", "r") as f:
    content = f.read()
    print(content)
# File auto-closes here. No f.close() needed!
```

 **Rule:** Always use `with open()` instead of manual `open()` / `close()`. It's safer, cleaner, and prevents resource leaks.

## Reading Methods Compared

| Method                     | Returns                   | Use When                 |
|----------------------------|---------------------------|--------------------------|
| <code>f.read()</code>      | Entire file as one string | Small files              |
| <code>f.readline()</code>  | One line at a time        | Processing line by line  |
| <code>f.readlines()</code> | List of all lines         | Need all lines as a list |

```
with open("data.txt", "r") as f:
    for line in f:          # Most memory-efficient way!
        print(line.strip()) # .strip() removes trailing \n
```

## File Encoding

Text files use **encoding** to convert characters to bytes. The default is usually UTF-8:

```
with open("data.txt", "r", encoding="utf-8") as f:  
    content = f.read()
```

💡 **Tip:** Always specify `encoding="utf-8"` to avoid issues with special characters (accents, emojis, non-English text).

## 9.4 Why Close Files?

---

-  Frees up system resources
  -  Ensures data is fully saved to disk
  -  Prevents file corruption
- 

## Practice Questions

---

1. Read `server_log.txt` and check if it contains the word `'Error'`.
  2. Write `update_highscore(new_score)` — only overwrite if new score is higher.
  3. Replace prohibited words ( `'donkey'` , `'idiot'` ) with `'#####'` in a text file.
  4. Wipe the contents of `temp_logs.txt` without deleting the file.
  5. Copy contents of `data.txt` into `data_backup.txt`.
  6. Count how many lines are in `notes.txt`.
-

# Chapter 10: Object Oriented Programming

---

## 10.1 What is OOP?

---

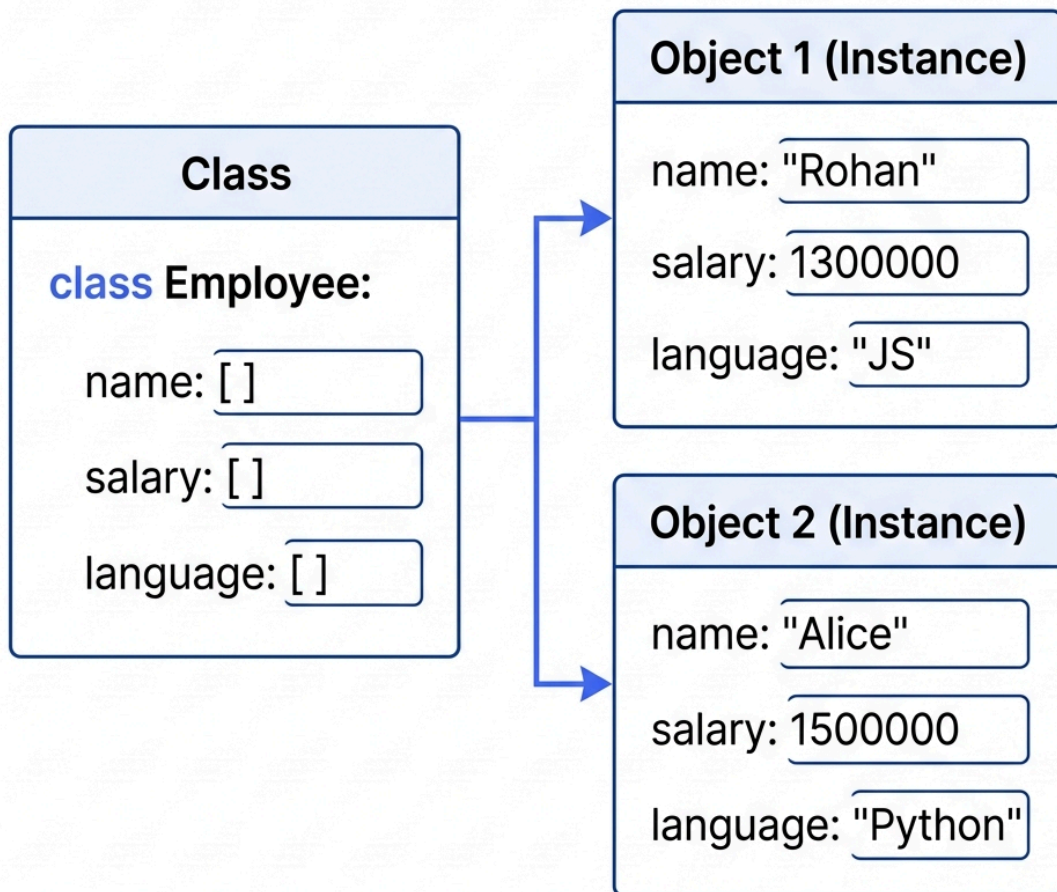
OOP organizes code into **objects** — bundles of related data and behavior. Instead of writing separate variables for every user, you create a **template (Class)** and reuse it.

### The 4 Pillars of OOP

| Pillar        | Meaning                                     | Example                                                                                |
|---------------|---------------------------------------------|----------------------------------------------------------------------------------------|
| Encapsulation | Bundle data + methods together              | A <code>BankAccount</code> class with <code>balance</code> and <code>withdraw()</code> |
| Abstraction   | Hide complex details, show simple interface | You call <code>car.start()</code> without knowing engine internals                     |
| Inheritance   | Child class reuses parent class code        | <code>Dog</code> inherits from <code>Animal</code>                                     |
| Polymorphism  | Same method name, different behavior        | <code>cat.speak()</code> → "Meow",<br><code>dog.speak()</code> → "Woof"                |

## 10.2 Class & Object

### Class (Blueprint) → Objects (Instances)



- **Class** = Blueprint (empty form template) — defines the structure
- **Object** = Instance (filled form with actual data) — a real thing made from the blueprint

You can create **unlimited objects** from one class — each with its own data.

```

class Employee:
    company = "Google"          # Class attribute (shared by ALL

emp1 = Employee()              # Create object
emp1.name = "Rohan"           # Instance attribute (unique to e

```

## Class vs Instance Attributes

- **Class attributes** → Shared by all objects. Defined outside `__init__`.  
Changed via `ClassName.attr`.
- **Instance attributes** → Unique to each object. Defined inside `__init__` with `self.attr`.

## 10.3 The `__init__` Constructor

Runs **automatically** when an object is created — used to initialize attributes. It's Python's version of a constructor.

```

class Employee:
    def __init__(self, name, salary):
        self.name = name          # Instance attribute
        self.salary = salary      # Instance attribute

e1 = Employee("Rohan", 50000)
e2 = Employee("Alice", 60000)
print(e1.name)                  # "Rohan" — each object has its own data

```



💡 **Key Point:** `self` refers to the **current object**. When you call `Employee("Rohan", 50000)`, Python automatically passes the new object as `self`.

💡 **Why `__init__`?** The double underscores ( `__` ) indicate it's a **dunder (magic) method** — Python calls it automatically. You never call `__init__()` directly.

## 10.4 Static Methods

Methods that **don't use `self`** — they belong to the class, not any specific object:

```
class Calculator:
    @staticmethod
    def add(a, b):
        return a + b

print(Calculator.add(5, 3))    # 8 – no object needed!
```

## Practice Questions

1. Create a `Programmer` class with `name`, `product`, `salary`. Create 2 objects.

2. Write a `Calculator` class with `square()`, `cube()`, and `sqrt()` methods.
  3. Create a class attribute `a` and instance attribute `a`. Do they conflict?
  4. Write a `Train` class with `bookTicket()`, `getStatus()`, and `getFare()`.
  5. Rename `self` to something else — does it still work?
-

# Chapter 11: Inheritance & More OOP

---

## 11.1 What is Inheritance?

---

A **child class** inherits all attributes and methods from a **parent class** — promoting code reuse. You write the common logic once in the parent and share it.

### Why Inheritance?

Imagine you have `Student`, `Teacher`, and `Staff` classes — they all share `name`, `email`, `get_id()`. Without inheritance, you'd copy-paste this code three times. With inheritance, you write it once in a `Person` class and inherit from it.

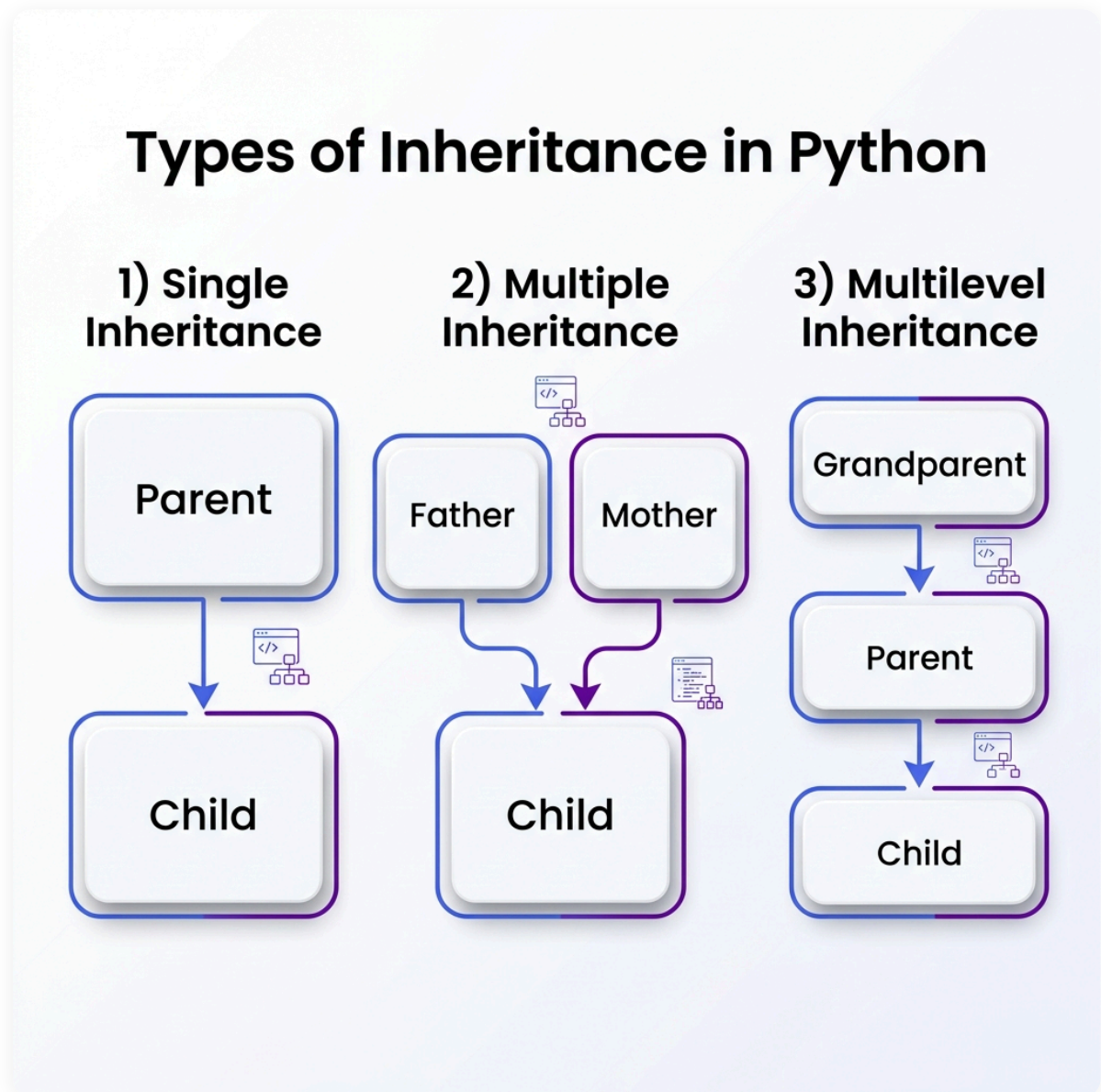
```
class Animal:
    def breathe(self):
        print("Breathing ... ")

class Dog(Animal):           # Inherits Animal
    def bark(self):
        print("Woof!")

d = Dog()
d.breathe()                 # Inherited from Animal!
d.bark()                    # Dog's own method
```

💡 **"is-a" Test:** Use inheritance when the child **is a** type of the parent. A Dog **is an** Animal ✅. A Wheel **is not** a Car ❌ (use composition instead).

## 11.2 Types of Inheritance



| Type   | Structure      |
|--------|----------------|
| Single | Parent → Child |

|                   |                              |
|-------------------|------------------------------|
| <b>Multiple</b>   | Parent1 + Parent2 → Child    |
| <b>Multilevel</b> | Grandparent → Parent → Child |

---

## Multiple Inheritance

```
class Father:
    def focus(self): print("Focus")

class Mother:
    def creativity(self): print("Creativity")

class Child(Father, Mother):    # Both parents!
    pass

c = Child()
c.focus()          # From Father
c.creativity()     # From Mother
```

## 11.3 The `super()` Method

Calls the **parent's method** — commonly used in `__init__` to initialize the parent class before adding child-specific attributes:

```
class Person:
    def __init__(self, name):
        self.name = name

class Student(Person):
    def __init__(self, name, grade):
        super().__init__(name)      # Initialize parent first
        self.grade = grade          # Then add child attribute

s = Student("Alice", "A")
print(s.name)      # "Alice" – from Person
print(s.grade)     # "A" – from Student
```

💡 **Why `super()`?** Without `super().__init__(name)`, the parent's `__init__` won't run and `self.name` won't be set. Always call `super()` first.

## 11.4 Method Overriding & Polymorphism

---

A child class can **redefine** a parent's method — this is called **overriding**:

```
class Animal:
    def speak(self):
        print("Some sound")

class Cat(Animal):
    def speak(self):          # Override parent method
        print("Meow!")

class Dog(Animal):
    def speak(self):          # Override parent method
        print("Woof!")

# Polymorphism – same method name, different behavior
for animal in [Cat(), Dog()]:
    animal.speak()           # "Meow!" then "Woof!"
```

💡 **Polymorphism** means "many forms" — the same method call produces different behavior depending on the object type.

## 11.5 Class Methods ( `@classmethod` )

Bound to the **class**, not the object. Uses `cls` instead of `self`:

```
class Employee:
    base_salary = 1000

    @classmethod
    def raise_salary(cls, amount):
        cls.base_salary += amount    # Changes for ALL objects
```

## @staticmethod vs @classmethod vs Regular Methods

| Type           | Gets <code>self</code> ? | Gets <code>cls</code> ? | Use When                  |
|----------------|--------------------------|-------------------------|---------------------------|
| Regular method | ✓ Yes                    | ✗ No                    | Needs object data         |
| @classmethod   | ✗ No                     | ✓ Yes                   | Needs class data          |
| @staticmethod  | ✗ No                     | ✗ No                    | Utility function in class |

## Practice Questions

1. Create `C2dVec` and inherit from it to create `C3dVec` using `super()`.
2. Build `ElectronicDevice` → `Phone` → `Smartphone`. Demonstrate multilevel inheritance.
3. Write an `Employee` class with a `salaryAfterIncrement` property.
4. Write a `Complex` class with an overloaded `+` operator ( `__add__` ).



# Chapter 12: Advanced Python

---

This chapter covers advanced features that will take your Python skills to the next level. These topics are commonly asked in **interviews** and used in **production code**.

## 12.1 Walrus Operator ( `:=` )

---

Assign and use a value **in the same expression** (Python 3.8+). Saves one line of code:

```
# Without walrus:
n = len("Python")
if n > 3:
    print(n)

# With walrus (shorter):
if (n := len("Python")) > 3:
    print(n)
```

## 12.2 Type Hints

---

Specify expected types for **better readability** (not enforced):

```
def add(a: int, b: int) → int:  
    return a + b
```

## 12.3 Match-Case (Python 3.10+)

---

A cleaner alternative to long `if-elif` chains:

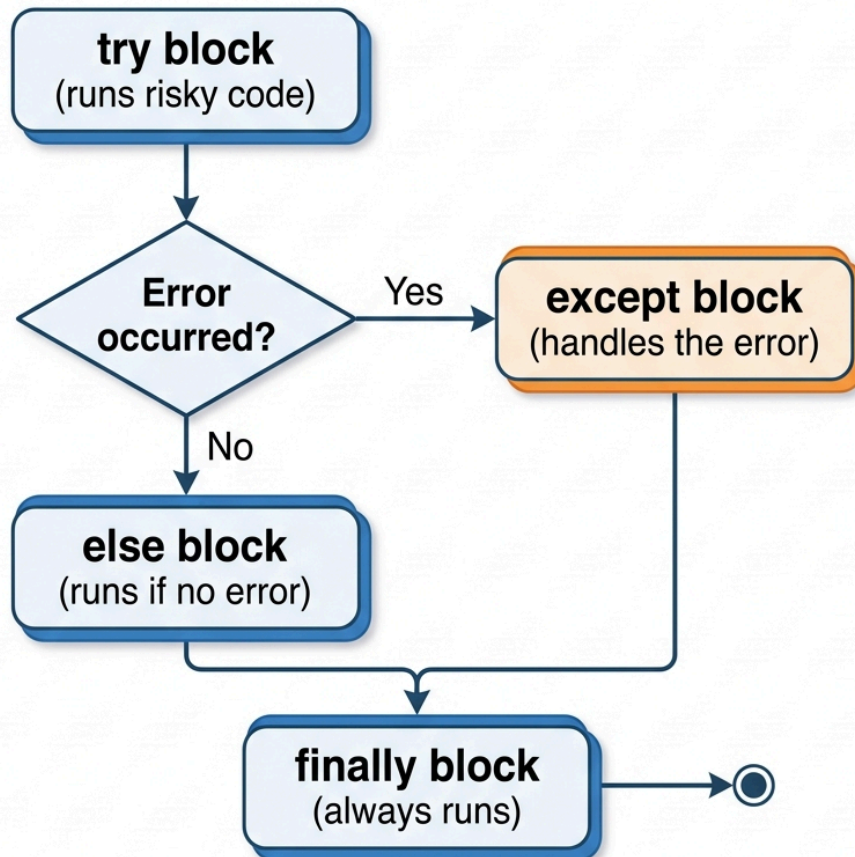
```
def day_type(day):  
    match day:  
        case "Saturday" | "Sunday":  
            return "Weekend"  
        case _:  
            return "Weekday"
```

## 12.4 Exception Handling

---

Catch errors **gracefully** instead of crashing. In production code, unhandled exceptions take down your entire app — `try/except` prevents that.

## EXCEPTION HANDLING FLOW



```
try:
    result = 10 / 0
except ZeroDivisionError:
    print("Cannot divide by zero!")
except ValueError as e:
    print(f"Bad value: {e}")    # 'as e' gives error details
finally:
    print("This always runs.")  # Cleanup code (close files,
```

## Common Built-in Exceptions

| Exception                      | When It Happens                             |
|--------------------------------|---------------------------------------------|
| <code>ValueError</code>        | Wrong value type: <code>int("hello")</code> |
| <code>TypeError</code>         | Wrong operation: <code>"5" + 5</code>       |
| <code>IndexError</code>        | Bad index: <code>[1,2][5]</code>            |
| <code>KeyError</code>          | Missing dict key: <code>d["missing"]</code> |
| <code>FileNotFoundError</code> | File doesn't exist                          |
| <code>ZeroDivisionError</code> | Division by zero                            |

| Block                | When It Runs                 |
|----------------------|------------------------------|
| <code>try</code>     | Always — contains risky code |
| <code>except</code>  | Only when error occurs       |
| <code>else</code>    | Only if NO error             |
| <code>finally</code> | <b>Always</b> — cleanup code |

## 12.5 List Comprehensions

Create lists in **one line**:

```
# Traditional:
squares = []
for x in range(5):
    squares.append(x**2)

# Comprehension:
squares = [x**2 for x in range(5)]

# With filter:
evens = [x for x in range(10) if x % 2 == 0]
```

## 12.6 Enumerate & Global

---

**Enumerate** — get index and value in a loop:

```
fruits = ["Apple", "Banana", "Cherry"]
for i, fruit in enumerate(fruits):
    print(f"{i}: {fruit}")
```

**Global** — modify an outer variable inside a function:

```
x = 10
def change():
    global x
    x = 20

change()
print(x)      # 20
```

⚠ **Warning:** Overusing `global` makes code hard to debug. Prefer **passing arguments** and **return values**.

---

## Practice Questions

---

1. Write a loop using the **walrus operator** that stops when user types `'quit'`.
  2. Create `get_average(nums: list) → float` with **type hints**.
  3. From `["Alice", "Bob", "Charlie", "Franklin"]`, get names with 5+ characters using **list comprehension**.
  4. Open `non_existent.txt` and handle `FileNotFoundError` gracefully.
  5. Write `http_error(code)` using **match-case** (200→"OK", 404→"Not Found").
-



# Congratulations!

You've completed the **Python Masterclass Handbook**. Here's what to do next.



## Career Paths You Can Pursue



Web Developer (Django/Flask)



Data Analyst / Scientist



AI / Machine Learning Engineer



Automation / DevOps Engineer



Game Developer (Pygame)



Cybersecurity Analyst



## Project Ideas to Build Now

|              |                                                          |
|--------------|----------------------------------------------------------|
| Beginner     | To-Do List CLI, Calculator, Password Generator, Quiz App |
| Intermediate | Expense Tracker, Weather App (API), Web Scraper, Chatbot |
| Advanced     | Blog with Django, Data Dashboard, ML Predictor, REST API |



## What to Learn Next

|                   |                                                                                                                                                        |
|-------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------|
| Web Development   | Django → <a href="https://djangoproject.com">djangoproject.com</a>   Flask → <a href="https://flask.palletsprojects.com">flask.palletsprojects.com</a> |
| Data Science      | NumPy, Pandas, Matplotlib → <a href="https://kaggle.com/learn">kaggle.com/learn</a>                                                                    |
| Machine Learning  | Scikit-learn, TensorFlow → <a href="https://tensorflow.org/tutorials">tensorflow.org/tutorials</a>                                                     |
| Practice Problems | LeetCode → <a href="https://leetcode.com">leetcode.com</a>   HackerRank → <a href="https://hackerrank.com">hackerrank.com</a>                          |

**Official Docs**

Python Docs → [docs.python.org/3](https://docs.python.org/3) | Real Python → [realpython.com](https://realpython.com)

---

"The only way to learn programming is to program."

— Dennis Ritchie

## **PYTHON MASTERCLASS HANDBOOK**

**by RahulSinghShah**